

Observability

A hands-on course in building agents by building their harness

Lesson 7

The terminal scrolled past, and the answers went with it

Lessons 5 and 6 made the harness safe to leave running. Now you run it where you can't watch it:

- Why did that take four minutes?
- Why did it cost that much?
- What did it run, and in what order?
- Which command did the guard refuse, and which did the box kill?

A record of what the harness did, kept as it does it

- Every call to the model
- Every tool it ran
- How long each took, and how many tokens each cost
- Every time a guardrail said no

A harness you can't see into is one you can't trust or fix.

Same run, two records, two readers

	Episode (Lesson 4)	Trace (Lesson 7)
Written for	the model	you, the person running quark
Holds	what working memory would have been	timings, token counts, exit codes, refusals
Used to	search its own past	answer your questions afterwards

A record for the model is context. A record for the operator is observability.

Built on control flow, and only control flow

- The model interface only knows about one call
- Output only knows about one tool
- The loop is where a call, its tool, the guard's decision and the next call all happen, in order
- So the loop is the one place a record of the run can be made

No new primitive: a production layer adds hardening.

Read off what's already there, and write it down

- **A trace.** One JSON line per thing that happened, appended in order
- **Timing.** Read the clock before and after a model call or a tool
- **Token accounting.** Every response says tokens in, out, and read from or written to the cache
- **Cost** is tokens times your provider's price; you keep the rates yourself

Observability only watches. It never changes what's sent, what runs, or when the loop stops.

`trace()`: one line per step, for whoever runs quark

Lesson 6's `quark.py` plus 14 lines. `time` joins the imports, and at the top of `# — control flow —` (abridged):

```
def trace(**event):  
    os.makedirs(".quark", exist_ok=True)  
    with open(".quark/traces.jsonl", "a") as f: f.write(json.dumps({...}) + "\n")
```

- Each line is `ts`, the `episode` this run is written to, and whatever you pass it
- Append-only: a run that crashes halfway still leaves everything before the crash
- `episode` ties the two records together

Then one call wherever something happens

```
trace(event="start", input=input)
    trace(event="stopped", steps=steps, tokens=spent)
    trace(event="too_long", drop=drop)
    trace(event="interrupted", during="saying")
        trace(event="refused", cmd=block.input["cmd"], why=no)
    trace(event="interrupted", during="acting")
```

- **start:** where a run begins and what came in
- **stopped:** the steps and tokens a limit stopped at
- **too_long:** the API said the prompt was too long, so compaction runs
- **refused:** the command the guard wouldn't run, and why
- **interrupted:** you pressed ESC, and whether quark was saying or doing something

Both read the clock before and after (abridged)

```
start = time.time()
with listening():
    response = call(...)
trace(event="model", seconds=round(time.time() - start, 2), ...)
```

```
start = time.time()
with listening():
    doing = subprocess.Popen(..., ...)
trace(event="tool", ..., seconds=round(time.time() - start, 2), ...)
```

- **model** also records `stop_reason` and four counts from `response.usage` :
input, output, cache read, cache write
- **tool** also records the command, characters printed and `exit` : non-zero if it failed, `137` if the box killed it

Run it: nobody there to answer the guard

```
quark.py "which file in this folder has the most lines? answer in one sentence" < /dev/null (shortened)
```

```
$ wc -l * 2>/dev/null | sort -rn | head -5
allow `wc -l * 2>/dev/null | sort -rn | head -5`? [y/N] [the person said no]
$ ls -la
-rw-r--r-- 1 root root 23893 Oct  6 20:30 big.txt
-rw-r--r-- 1 root root      8 Oct  6 20:30 small.txt

$ wc -l big.txt small.txt
5000 big.txt
   2 small.txt
5002 total

`big.txt` has the most lines, at 5,000, compared with 2 in `small.txt`.
```

What you see afterwards, in `.quark/traces.jsonl`

event	seconds	stop_reason	in	out	cache_read	cache_write
model	1.37	tool_use	95	70	0	7770
model	1.66	tool_use	180	51	7770	0
tool <code>ls -la</code>	0.12	exit 0				
model	1.94	tool_use	397	93	7770	0
tool <code>wc -l big.txt small.txt</code>	0.09	exit 0				
model	1.74	end_turn	518	34	7770	0

The trace's fields as a table; the `start` line and the `refused` line are left out.

One JSON object per line, so you can ask it questions

Add up one run's model calls, by its episode, with `jq`:

```
{"calls":4,"input":1190,"output":248,"cache_read":23310,"cache_write":7770,"seconds":6.7100000000000001}
```

The whole run: four calls, about 6.7 seconds of model time, and most of what it read came from the cache.

A run where things go wrong

A copy with the sandbox's `TIMEOUT` lowered to 5, and `y` piped in for the guard (shortened):

```
$ ls /nonexistent
ls: cannot access '/nonexistent': No such file or directory

$ sleep 60
allow `sleep 60`? [y/N]
(killed: ran over 5 seconds or out of memory)
```

The agent told you what happened, this time. Without the trace, you'd only know if you were watching.

Everything that didn't go to plan

```
jq -c 'select((.event=="tool" and .exit!=0) or .event=="refused")' .quark/traces.jsonl
```

event	cmd	seconds	exit / why
refused	<code>wc -l * 2>/dev/null sort -rn head -5</code>		the person said no
tool	<code>ls /nonexistent</code>	0.12	2
tool	<code>sleep 60</code>	5.1	137

The trace's fields as a table, across both runs.

Press ESC, and the trace says why it stopped

In a terminal: `sleep 30`, `y` to the guard, ESC three seconds later. The trace, with only the fields that matter here:

```
{"event": "start"}  
{"event": "model", "seconds": 1.72}  
{"event": "tool", "cmd": "sleep 30", "seconds": 3.23, "exit": 137}  
{"event": "interrupted", "during": "acting"}  
{"event": "model", "seconds": 2.01}
```

- Stopped at 3.23 seconds: `exit 137`, the box's kill
- The next line says why: you interrupted it while it was acting
- Then one more call, for the model to acknowledge it
- The trace knows more about that command than the model does

What else observability can be

- **What's recorded:** events, *spans* with a parent (a tree), or every full request and response
- **Where it goes:** a file, the terminal, a database, an OpenTelemetry collector
- **When you see it:** afterwards, a live line per step, or a dashboard that alerts you
- **What's worked out:** tokens, cost, latency, cache hit rate, tool failures, steps per task
- **How you ask and how long it's kept:** `grep` and `jq`, SQL or a trace UI; sampling, rotation, deletion

A product on its own, and a fuller example

Tracing platforms like Langfuse and LangSmith are this layer: you send them spans, they store them, price them and show them as a tree.

`observability.py` , built on Lesson 3's `control_flow.py` , does more of that:

- **Spans:** `span()` records an id, a `parent` , a `start` and `seconds` ; a crash is recorded and still crashes
- **Cost:** `cost()` multiplies the four token counts by `PRICE` (example rates)
- **A live line** per step, **a summary** per run, and **a report** across every run
- **A second opinion:** Jev, a small decision model, says whether each tool failed

A live line per step, while it runs

`observability.py "run ls /nonexistent, then run sleep 3 ..."` prints a line per step (shortened):

```
[step 1: model 2.76s, 397 tokens in, 233 out, $0.0047]
$ ls /nonexistent
[step 1: tool 0.0s, exit 2, Jev: failed 0.98]
$ sleep 3
[step 1: tool 3.0s, exit 0, Jev: failed 0.02]
[step 2: model 0.97s, 712 tokens in, 52 out, $0.0029]
```

At the end, `row()` sums the run into one line: `prob.` 1 (one tool exited non-zero), `jev` 1, 6.7 seconds, 3.0 of them the `sleep`.

A second opinion: did that tool fail? (abridged)

```
def failed(cmd, result):  
    try:  
        return round(jev.system_one({"command": cmd, "result": result}, ERROR).noul["error"].noul, 2)  
    except Exception:  
        return None
```

- `ERROR` asks Jev: does `result` show that the command failed or hit an error?
- The answer goes in its own `jev` span, child of the tool's, beside the exit code
- The `jev` column counts tools Jev was at least `SURE` (0.9) sure failed
- Asking is model interface; writing it down is observability. It never changes a run

Where the exit code and Jev disagree

```
observability.py "run grep -rn FIXME ..., then run ls /nonexistent |  
head -1, ..." (shortened):
```

```
$ grep -rn FIXME production/07-observability  
[step 1: tool 0.0s, exit 1, Jev: failed 0.76]  
$ ls /nonexistent | head -1  
ls: cannot access '/nonexistent': No such file or directory  
[step 1: tool 0.0s, exit 0, Jev: failed 0.97]
```

The exit code counts the `grep` that found nothing. Jev counts the `ls` the pipe hid. Jev still gave the `grep` 0.76: a probability, not a fact, so it sits beside the exit code.

A table of every run says what's normal

`observability.py report` reads the whole log: one line per run (some columns left out).

run	calls	tools	prob.	jev	time	cost
0348969a	2	1	0	0	2.2s	\$0.0045
ff40e40a	4	3	0	0	7.4s	\$0.0228
6dae6b62	2	2	1	1	6.7s	\$0.0076
c0808f2d	2	2	1	1	2.6s	\$0.0058

The run that went looking for a file with the wrong name took four calls and cost about five times the first.

The rule

Record what the harness does as it does it: each model call, each tool, each refusal, how long it took and what it cost, in a log you can search afterwards.

Do it where control flow already sees the whole sequence, read from what's already there, and keep it apart from what the model remembers.

What observability never does

- **Control flow:** it sits in the loop, but never decides what runs next or when to stop
- **Model interface:** sends and receives exactly as before; it only reads `usage`
- **Input:** it gathers none
- **Context:** the trace is not in the request; that's what makes it observability
- **Output:** runs tools the way it always did, only with a clock around them

What's missing: it watches, and that's all

It will faithfully record that:

- the API dropped a call halfway through a long run
- the model was cut off in the middle of a command
- you pressed ESC, and everything it had said or printed up to then was thrown away
- the process died and took the work with it

It can tell you exactly where things broke; it can't pick up from there. A harness that runs unattended has to survive a bad day, not just describe one.

Next: Resilience

Lesson 8